

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: FPGA Based System Design (01CT0718)	Aim: To design and implement a 8-bit RISC processor architecture using HDL and FPGA.	
Experiment No: 12	Date:	Enrolment No: 92301733025

Aim: To design and implement a 8-bit RISC processor architecture using HDL and FPGA.

Software required:

Vivado Simulator

Theory:

A RISC (Reduced Instruction Set Computer) processor is a processor architecture based on a small and simple set of instructions. RISC processors generally use simple instructions that can be executed efficiently with a regular instruction format.

An 8-bit RISC processor operates on 8-bit data and is designed using components such as:

- Program Counter (PC)
- Instruction Memory
- Instruction Register
- Register File
- Arithmetic Logic Unit (ALU)
- Control Unit
- Data Memory
- Multiplexer
- Output/Register logic

The Program Counter (PC) stores the address of the next instruction to be executed. Normally, it is incremented after fetching an instruction.

The Instruction Memory stores the program instructions. The instruction pointed to by the PC is fetched and loaded into the instruction register.

The Control Unit decodes the instruction and generates control signals required by different processor components.

The Register File contains a small number of registers used to store operands and intermediate results. RISC architectures make extensive use of registers because register-based operations are fast.

The ALU (Arithmetic Logic Unit) performs arithmetic and logical operations such as:

- Addition
- Subtraction
- AND
- OR
- XOR
- Comparison

The Data Memory is used for storing and loading data when required by memory-access instructions.

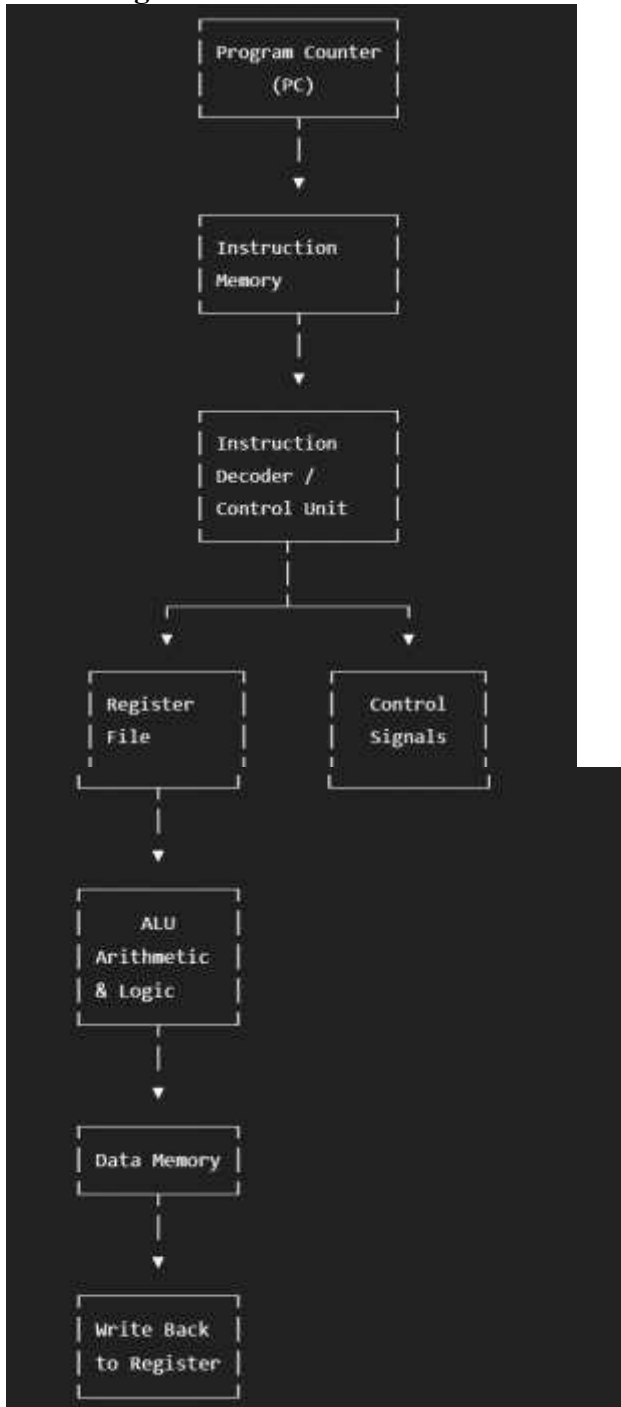
A basic RISC processor generally follows the instruction execution sequence:

Fetch → Decode → Execute → Memory → Writeback

For a simple non-pipelined processor, these operations may be performed sequentially for each instruction. The processor can be implemented using Verilog HDL and its operation can be verified using Vivado simulation. The simulation waveform can be used to observe the clock, reset, program counter, instruction, ALU operation, register values, and processor output.

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: FPGA Based System Design (01CT0718)	Aim: To design and implement a 8-bit RISC processor architecture using HDL and FPGA.	
Experiment No: 12	Date:	Enrolment No: 92301733025

Block diagram:



The PC provides the instruction address. The instruction memory supplies the instruction to the decoder. The control unit generates the required control signals. Operands are obtained from the register file and processed by the ALU. Memory operations access the data memory, and the resulting data is written back to the register file.

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: FPGA Based System Design (01CT0718)	Aim: To design and implement a 8-bit RISC processor architecture using HDL and FPGA.	
Experiment No: 12	Date:	Enrolment No: 92301733025

Truth Table:

For the simulation, we'll use a small instruction set so that the processor can be easily verified.

We'll define four instructions:

Opcode	Instruction	Operation
0000	ADD	$R1 = R2 + R3$
0001	SUB	$R1 = R2 - R3$
0010	AND	$R1 = R2 \& R3$
0011	OR	$R1 = R2 R3$

ALU Operation Table :

Opcode	Operation	ALU Output
0000	ADD	$A + B$
0001	SUB	$A - B$
0010	AND	$A \& B$
0011	OR	$A B$

Control Table:

Opcode	ALU Operation	Register Write
0000	ADD	1
0001	SUB	1
0010	AND	1
0011	OR	1

This gives us a simple instruction set that is sufficient to demonstrate the operation of an 8-bit RISC processor.

Boolean Expression:

For an ALU-based RISC processor, the operation is selected according to the opcode.

For example:

ADD

$$Y = A + B$$

SUB

$$Y = A - B$$

AND

$$Y = A \cdot B$$

OR

$$Y = A + B$$

where the OR operation is logically represented as:

$$Y = A \text{ OR } B$$

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: FPGA Based System Design (01CT0718)	Aim: To design and implement a 8-bit RISC processor architecture using HDL and FPGA.	
Experiment No: 12	Date:	Enrolment No: 92301733025

The register-write control can be represented as:

$$RegWrite = 1$$

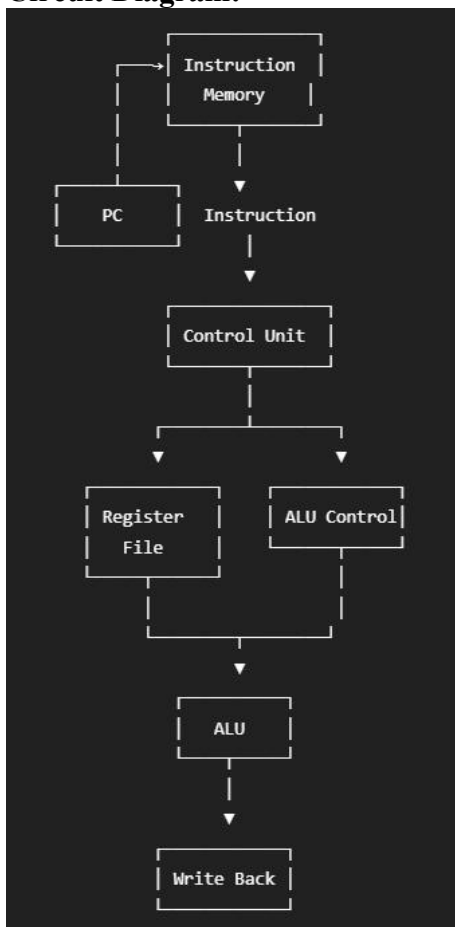
for arithmetic and logical instructions.

The program counter normally follows:

$$PC_{next} = PC + 1$$

for sequential instruction execution.

Circuit Diagram:



Verilog Code:

Design Module :

```

module risc_8bit
(
  input wire clk,
  input wire reset,
  output reg [7:0] result,
  output reg [7:0] pc
);

reg [7:0] registers [0:3];
reg [7:0] instruction_memory [0:15];

```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: FPGA Based System Design (01CT0718)	Aim: To design and implement a 8-bit RISC processor architecture using HDL and FPGA.	
Experiment No: 12	Date:	Enrolment No: 92301733025

```

reg [7:0] instruction;

reg [3:0] opcode;
reg [1:0] rd;
reg [1:0] rs1;
reg [1:0] rs2;

integer i;

always @(posedge clk) begin

    if (reset) begin

        pc <= 8'd0;
        result <= 8'd0;

        for (i = 0; i < 4; i = i + 1)
            registers[i] <= 8'd0;

    end

    else begin

        // Fetch instruction
        instruction = instruction_memory[pc];

        // Decode instruction
        opcode = instruction[7:4];
        rd     = instruction[3:2];
        rs1    = instruction[1:0];

        // Execute
        case (opcode)

            // ADD
            4'b0000: begin
                registers[rd] <= registers[rs1] + registers[0];
                result <= registers[rs1] + registers[0];
            end

            // SUB
            4'b0001: begin
                registers[rd] <= registers[rs1] - registers[0];
                result <= registers[rs1] - registers[0];
            end

            // AND
            4'b0010: begin

```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: FPGA Based System Design (01CT0718)	Aim: To design and implement a 8-bit RISC processor architecture using HDL and FPGA.	
Experiment No: 12	Date:	Enrolment No: 92301733025

```

        registers[rd] <= registers[rs1] & registers[0];
        result <= registers[rs1] & registers[0];
    end

```

```

    // OR
    4'b0011: begin
        registers[rd] <= registers[rs1] | registers[0];
        result <= registers[rs1] | registers[0];
    end

```

```

    default: begin
        result <= 8'd0;
    end

```

```

endcase

```

```

    // Increment program counter
    pc <= pc + 1'b1;

```

```

end

```

```

end

```

```

endmodule

```

```

    module risc_8bit
    ( input wire clk,
      input wire reset,
      output reg [7:0] result,
      output reg [7:0] pc
    );

```

```

    reg [7:0] registers [0:3];
    reg [7:0] instruction_memory [0:15];

```

```

    reg [7:0] instruction;

```

```

    reg [3:0] opcode;
    reg [1:0] rd;
    reg [1:0] rs1;
    reg [1:0] rs2;

```

```

    integer i;

```

```

    always @(posedge clk) begin

```

```

        if (reset) begin

```

```

            pc <= 8'd0;

```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: FPGA Based System Design (01CT0718)	Aim: To design and implement a 8-bit RISC processor architecture using HDL and FPGA.	
Experiment No: 12	Date:	Enrolment No: 92301733025

```
result <= 8'd0;
```

```
for (i = 0; i < 4; i = i + 1)
    registers[i] <= 8'd0;
```

```
end
```

```
else begin
```

```
    // Fetch instruction
    instruction = instruction_memory[pc];
```

```
    // Decode instruction
    opcode = instruction[7:4];
    rd     = instruction[3:2];
    rs1    = instruction[1:0];
```

```
    // Execute
    case (opcode)
```

```
        // ADD
        4'b0000: begin
            registers[rd] <= registers[rs1] + registers[0];
            result <= registers[rs1] + registers[0];
        end
```

```
        // SUB
        4'b0001: begin
            registers[rd] <= registers[rs1] - registers[0];
            result <= registers[rs1] - registers[0];
        end
```

```
        // AND
        4'b0010: begin
            registers[rd] <= registers[rs1] & registers[0];
            result <= registers[rs1] & registers[0];
        end
```

```
        // OR
        4'b0011: begin
            registers[rd] <= registers[rs1] | registers[0];
            result <= registers[rs1] | registers[0];
        end
```

```
        default: begin
            result <= 8'd0;
        end
```

```
endcase
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: FPGA Based System Design (01CT0718)	Aim: To design and implement a 8-bit RISC processor architecture using HDL and FPGA.	
Experiment No: 12	Date:	Enrolment No: 92301733025

```
// Increment program counter
pc <= pc + 1'b1;
```

```
end
```

```
end
```

```
endmodule
```

TESTBENCH :

```
module risc_8bit_tb;
```

```
reg clk;
reg reset;
```

```
wire [7:0] result;
wire [7:0] pc;
```

```
risc_8bit uut (
    .clk(clk),
    .reset(reset),
    .result(result),
    .pc(pc)
);
```

```
// Clock generation
always #5 clk = ~clk;
```

```
initial begin
```

```
    clk = 0;
    reset = 1;
```

```
// Initialize registers
uut.registers[0] = 8'd10;
uut.registers[1] = 8'd20;
uut.registers[2] = 8'd30;
uut.registers[3] = 8'd40;
```

```
// ADD
uut.instruction_memory[0] = 8'b00000100;
```

```
// SUB
uut.instruction_memory[1] = 8'b00010100;
```

```
// AND
uut.instruction_memory[2] = 8'b00100100;
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: FPGA Based System Design (01CT0718)	Aim: To design and implement a 8-bit RISC processor architecture using HDL and FPGA.	
Experiment No: 12	Date:	Enrolment No: 92301733025

```
// OR
uut.instruction_memory[3] = 8'b00110100;

#10;

reset = 0;

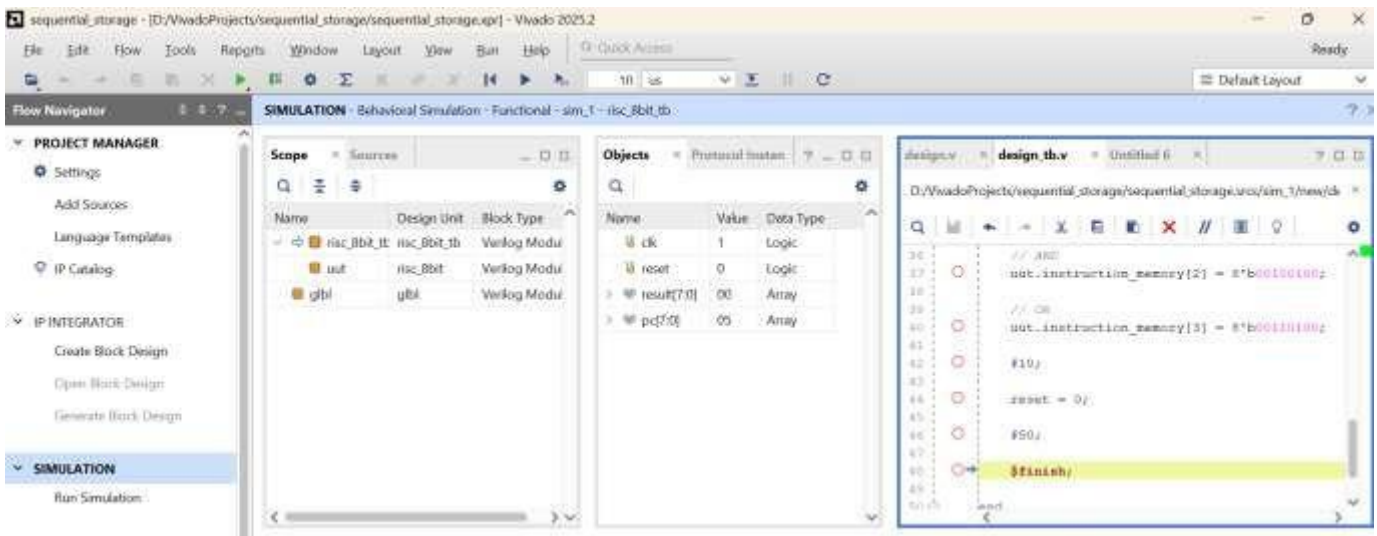
#50;

$finish;

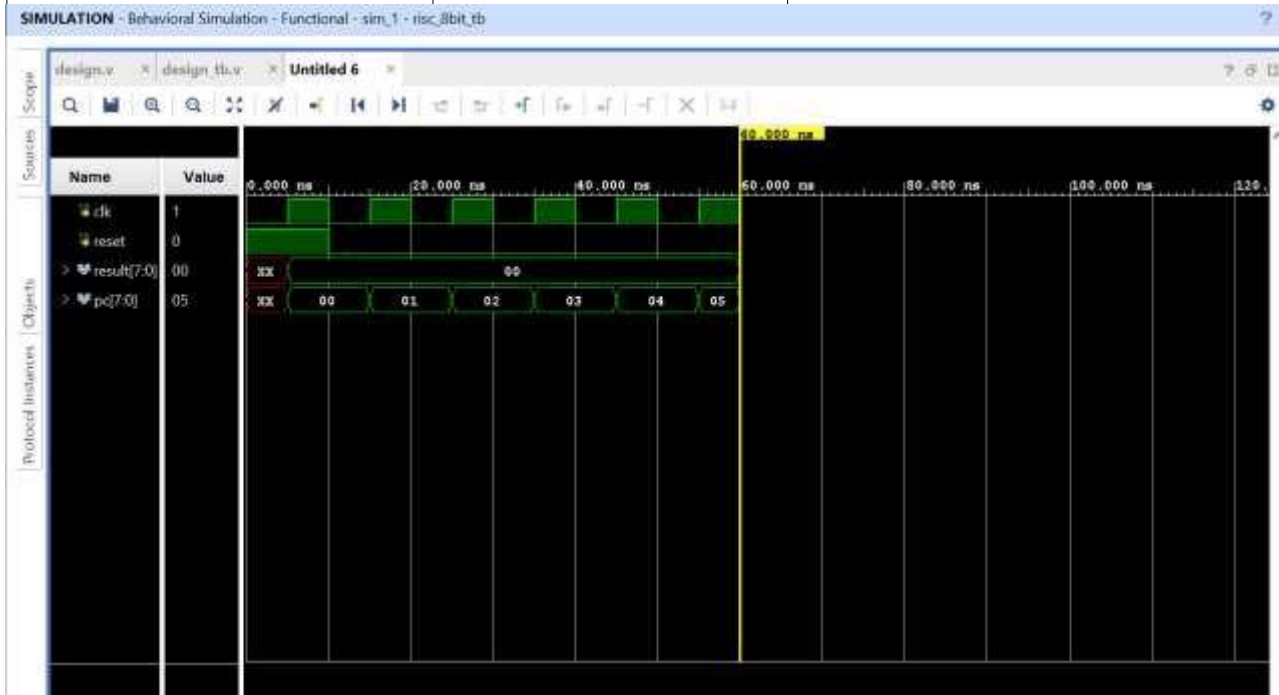
end

endmodule
```

Simulation Results:



 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: FPGA Based System Design (01CT0718)	Aim: To design and implement a 8-bit RISC processor architecture using HDL and FPGA.	
Experiment No: 12	Date:	Enrolment No: 92301733025



The simulation waveform verifies the operation of the 8-bit RISC processor. The program counter increments sequentially to fetch instructions from the instruction memory. The instruction opcode determines the ALU operation, and the corresponding result is generated and stored in the destination register. The simulation confirms the basic instruction fetch, decode, execution, and write-back operations of the processor.

Conclusion:

The 8-bit RISC processor architecture was successfully designed using Verilog HDL and verified through Vivado simulation. The basic processor components including the program counter, instruction memory, register file, control logic, and ALU were implemented. The simulation waveform verified instruction fetching, program counter operation, ALU execution, and result generation. Thus, the functionality of the basic 8-bit RISC processor architecture was successfully demonstrated through simulation.

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: FPGA Based System Design (01CT0718)	Aim: To design and implement a 8-bit RISC processor architecture using HDL and FPGA.	
Experiment No: 12	Date:	Enrolment No: 92301733025

Post Lab Exercise:

1. Why is a RISC architecture preferred over a CISC architecture for FPGA implementation?
RISC uses a smaller and simpler instruction set, which makes the control logic easier to implement and can reduce hardware complexity.
2. How does a reduced instruction set improve processor performance?
Simple instructions generally require less decoding and execution complexity, allowing instructions to be executed efficiently.
3. Why is a register file essential in an 8-bit RISC processor?
Simple instructions generally require less decoding and execution complexity, allowing instructions to be executed efficiently.
4. How does the control unit influence processor operation?
The control unit decodes instructions and generates control signals that determine how the registers, ALU, memory, and other processor components operate.
5. Why is the program counter (PC) necessary in processor architecture?
The PC stores the address of the next instruction to be fetched and allows instructions to be executed in the correct sequence.
6. How does instruction decoding affect processor performance?
Instruction decoding determines which operation needs to be performed and generates the appropriate control signals. Efficient decoding can reduce execution delay.
7. Why is an Arithmetic Logic Unit (ALU) the core component of a RISC processor?
The ALU performs the arithmetic and logical operations required by processor instructions, making it a fundamental execution unit.
8. How does pipelining improve processor throughput?
The ALU performs the arithmetic and logical operations required by processor instructions, making it a fundamental execution unit.
9. Why is hazard detection important in a pipelined RISC processor?
Hazard detection prevents incorrect execution when instructions have dependencies or conflicts that could cause incorrect results.
10. How does HDL simulation improve processor design reliability?
HDL simulation allows the processor's functionality and timing behavior to be tested before hardware implementation. Errors can be detected and corrected early in the design process.